

NewsDigest: News Collection and Automatic Summarisation

COMP4020 — NLP Final Project Report (Phase 2)

Team: Nguyen Hoang Hieu (Ethan), Thai Ba Hung, Nguyen Quoc Dang, Le Nguyen Gia Binh Emails: 22hieu.nh@vinuni.edu.vn · 22hung.tb@vinuni.edu.vn · 22dang.nq@vinuni.edu.vn · 22binh.lng@vinuni.edu.vn GitHub: https://github.com/dangnguyen271/NLP_FinalProject

1. Introduction

The growth of digital news has made it increasingly difficult for readers to keep up with current events. News websites publish a large number of articles every day, and many are lengthy or repetitive across sources, so readers spend significant time identifying the most relevant information. NewsDigest is an NLP-based prototype that collects English news articles and automatically generates concise summaries. The main NLP task is **text summarisation**, and the application domain is **digital news media**.

The system is designed as a practical university-level pipeline that combines data collection, preprocessing, summarisation, evaluation, and a small web prototype. Three research questions guide the work:

- **RQ1** — How effectively can an NLP system summarise newspaper articles in a way that is concise, readable, and informative?
- **RQ2** — Does an abstractive transformer (BART) materially outperform extractive baselines (Lead-3, TextRank) in ROUGE on CNN/DailyMail and XSum?
- **RQ3** — Which article characteristics (length, source, writing style) drive summary-quality differences between methods?

We use three datasets. The **bundled sample** ([data/news_sample.csv](#), 24 article-summary pairs) is included in the repository for offline tests and CI. The **CNN/DailyMail v3.0.0** dataset (Hermann et al. 2015; See et al. 2017) is the primary benchmark; **XSum** (Narayan et al. 2018) is used as a secondary benchmark with extreme single-sentence summaries. Both benchmark datasets are fetched on demand by [scripts/fetch_datasets.py](#) from the Hugging Face Datasets hub. A handful of recent articles are scraped at presentation time from public URLs through the [scrape](#) CLI command.

Three challenges shape the methodology. **Long-form inputs**: CNN/DailyMail articles regularly exceed BART's 1024-token cap, requiring truncation or chunking. **Webpage noise**: scraped articles include HTML, ads, share buttons, and "read more" boilerplate that must be cleaned. **Style heterogeneity**: CNN, BBC, and blog posts differ in lead structure, average length, and vocabulary.

2. Methodology

2.1 Pipeline overview

The pipeline is implemented in [src/nlp_project/](#) and exposed through one CLI:

```
python -m nlp_project.cli run-all --config config/project_config.yaml
```

It validates the CSV, splits 75/25 train/test, runs every available summarisation method on the test split, scores ROUGE against the reference summaries, produces five report figures, and regenerates the proposal markdown and PDF.

2.2 Preprocessing

`nlp_project.preprocess` performs three deterministic steps:

1. `strip_html` removes `<script>`, `<style>`, `<iframe>`, and other non-content tags using BeautifulSoup (`lxml` parser).
2. `normalize_article` applies Unicode normalisation (NFKC), drops control characters, strips boilerplate phrases (*"Share this article"*, *"click here to subscribe"*, copyright lines), and collapses whitespace. Crucially, it preserves casing and punctuation — both signals that downstream summarisers rely on for fluency.
3. `split_sentences` is a lightweight regex sentence splitter that handles common abbreviations (*Dr.*, *U.S.*, *e.g.*) without needing the NLTK punkt download.

2.3 Summarisation methods

Three methods share the same input contract:

Method	Type	Implementation
<code>lead_3</code>	Extractive	First three sentences (the strongest single baseline on CNN/DailyMail).
<code>textrank</code>	Extractive	TF-IDF + cosine-similarity graph + <code>networkx.pagerank</code> .
<code>bart</code>	Abstractive	<code>facebook/bart-large-cnn</code> via the Hugging Face <code>summarization</code> pipeline.

BART is loaded lazily and gated behind `model.use_abstractive: true` in the YAML config; it requires the `[transformer]` optional install. The first two work entirely offline.

2.4 Evaluation protocol

We score every generated summary against the gold reference using ROUGE-1, ROUGE-2, ROUGE-L, and ROUGE-Lsum (Lin 2004), computed by `rouge-score` with Porter stemming. The pipeline writes:

- `reports/rouge_metrics.json` — per-method mean ROUGE scores.
- `reports/method_summary.csv` — one row per method with ROUGE, mean summary length, mean compression ratio, mean inference time.
- `reports/qualitative_review.csv` — every test-set row × method with the generated summary, reference, ROUGE breakdown — the working dataset for the qualitative review.

2.5 Reproducibility

All randomness is seeded from a single `project.random_seed` value in the YAML config. Every artefact under `reports/` is regenerable from a fresh `git clone` with `python -m nlp_project.cli run-all`. The pytest suite (26 tests) covers config, data, preprocessing, summarisation, evaluation, visualisations, the proposal generator, and the CLI; it runs in ~4 seconds, fully offline.

3. Results

The numbers below are produced by `python -m nlp_project.cli run-all` on the bundled 24-row sample (18 train / 6 test). When the team runs `scripts/fetch_datasets.py --dataset cnn_dailymail` and points the config at the downloaded CSV, the same command regenerates this section against the canonical benchmark.

3.1 RQ1 — Lead-3 baseline

On the held-out test split of the bundled sample:

Metric	Value
ROUGE-1 F1	0.359
ROUGE-2 F1	0.121
ROUGE-L F1	0.257
Mean summary	69 tokens
Compression	82.7%

Lead-3 is the most reliable single baseline in news summarisation because news lead paragraphs are *engineered* to be summaries. Even on this small sample, ROUGE-1 ≈ 0.36 is competitive with published TextRank results on CNN/DailyMail. The remaining gap to human references comes from the bundled sample's preference for very short one-sentence highlights, while Lead-3 returns three full sentences.

3.2 RQ2 — Method comparison

Method	ROUGE-1	ROUGE-2	ROUGE-L	Mean tokens	Compression	Inference (s)
lead_3	0.359	0.121	0.257	69	82.7%	~0.000
textrank	0.367	0.128	0.269	69	83.0%	0.007

TextRank edges out Lead-3 by ~1 point of ROUGE-1 on the bundled sample. On CNN/DailyMail, published results put `bart-large-cnn` ahead of both extractive baselines by roughly 10-15 points of ROUGE-1 (~0.44 vs. ~0.30), so we expect the gap to widen significantly once BART is enabled. The fact that two extractive methods are this close on the bundled sample is itself informative: it tells us that on short news with a clear lead-paragraph structure, extractive baselines are very hard to beat without a learned model.

Inference time is negligible for both extractive methods (sub-10 ms per article). BART, when enabled, takes ~1-3 seconds per article on CPU and is the dominant compute cost in the pipeline.

3.3 RQ3 — Where the methods differ

Reviewing [reports/qualitative_review.csv](#) reveals three patterns:

1. **Long articles → identical outputs.** When an article has fewer than four sentences, Lead-3 returns the whole article and TextRank degrades to the same. Compression ratios stay above 80%.
2. **Multiple-subject articles → TextRank wins.** When the article has multiple subjects (e.g. announcement + follow-up + future plans), TextRank's central-sentence ranking picks the most globally connected sentence, while Lead-3 may miss the most important content by sticking to position one.
3. **One-sentence references hurt both methods.** XSum-style single-sentence references penalise any multi-sentence extractive output. The qualitative CSV shows several rows where the generated summary is faithful and readable but ROUGE-2 collapses because of length mismatch.

The compression-ratio plot ([reports/figures/compression_ratio.png](#)) makes (1) visually obvious: the bars sit near 80% for both extractive methods. The per-example ROUGE-1 distribution plot ([reports/figures/per_example_rouge.png](#)) shows that the methods are tightly correlated row-by-row, confirming (2) and (3).

3.4 Article length distribution

The article length histogram ([reports/figures/article_length.png](#)) shows the bundled sample is tightly concentrated between 60 and 110 tokens. Real CNN/DailyMail articles average ~780 tokens and stress BART's 1024-token cap; we discuss this length mismatch under Limitations.

3.5 Descriptive dataset statistics

The bundled development dataset is deliberately small, but it is balanced enough to exercise the whole pipeline and expose length/compression behaviour before switching to the larger benchmarks. Validation reports zero duplicated article-summary pairs.

Statistic	Article tokens	Reference-summary tokens	Article sentences	Reference compression
Count	24	24	24	24
Mean	81.4	23.6	3.7	29.1%
Median	80.0	23.0	4.0	29.0%
Standard deviation	8.6	3.0	0.6	3.0 pp
Minimum	67	19	3	22.9%
25th percentile	76.0	21.0	3.0	27.5%
75th percentile	83.3	25.3	4.0	30.4%
Maximum	105	31	5	36.0%

The deterministic 75/25 split gives 18 training rows and 6 test rows. The split is not materially length-skewed: the training rows average 80.5 article tokens and 23.4 reference tokens, while the held-out test rows average 84.0 article tokens and 24.2 reference tokens. Reference compression is also similar across

splits (29.2% train vs. 28.7% test), so the reported ROUGE scores are not inflated by an unusually short or easy test set.

3.6 Per-example ROUGE distribution

Means alone hide a lot of useful behaviour on a six-row test set. The table below summarises the per-example distribution in [reports/qualitative_review.csv](#).

Method	Metric	Mean	Median	Std. dev.	IQR	Min	Max
Lead-3	ROUGE-1 F1	0.359	0.353	0.091	0.296-0.432	0.242	0.472
Lead-3	ROUGE-2 F1	0.121	0.096	0.058	0.081-0.163	0.067	0.207
Lead-3	ROUGE-L F1	0.257	0.269	0.047	0.242-0.278	0.176	0.315
TextRank	ROUGE-1 F1	0.367	0.372	0.093	0.305-0.432	0.242	0.484
TextRank	ROUGE-2 F1	0.128	0.134	0.048	0.089-0.166	0.067	0.180
TextRank	ROUGE-L F1	0.269	0.271	0.065	0.242-0.286	0.176	0.374

TextRank improves the mean by 0.008 ROUGE-1, 0.006 ROUGE-2, and 0.012 ROUGE-L. However, the paired comparison is mixed: TextRank is better on 1 of 6 examples, Lead-3 is better on 1 of 6 examples, and the two methods tie on 4 of 6 examples. This supports the qualitative observation that both extractive systems often choose the same lead-heavy sentences on short news articles.

The strongest Lead-3 row is the offshore wind-farm article ([id=17](#), ROUGE-1 F1 0.472); the weakest is the Hanoi-Sydney flight article ([id=9](#), ROUGE-1 F1 0.242). TextRank's best row is the education-chatbot article ([id=1](#), ROUGE-1 F1 0.484), where graph centrality selects a sentence closer to the reference than the strict lead position.

3.7 Precision, recall, and length diagnostics

The extractive methods have much higher recall than precision because they output three full sentences while the references are concise one-sentence highlights. This is visible in the mean ROUGE component scores:

Method	ROUGE-1 precision	ROUGE-1 recall	ROUGE-2 precision	ROUGE-2 recall	ROUGE-L precision	ROUGE-L recall
Lead-3	0.243	0.691	0.081	0.240	0.175	0.492
TextRank	0.252	0.697	0.087	0.249	0.185	0.508

Generated summaries average 68.7 tokens for both methods, compared with 24.2 tokens for the test references. As a result, the generated-summary compression ratio is about 83%, whereas the gold-reference compression ratio is about 29%. This length mismatch explains why recall is strong while precision and ROUGE-2 remain modest: the generated summaries usually contain the correct topic but include many extra tokens not present in the human highlight.

Runtime remains suitable for an interactive demo. Lead-3 is effectively instant (mean runtime below 0.1 ms per article in this run). TextRank averages 13.8 ms per article, with a median below 1 ms and one

slower row caused by graph construction. BART is expected to dominate runtime once enabled, so it should be presented as a quality-oriented option rather than the default low-latency mode.

3.8 XSum snapshot statistics

The team also fetched a 500-row XSum test snapshot with:

```
python scripts/fetch_datasets.py --dataset xsum --split test --max-rows 500
```

This snapshot contains zero empty articles, zero empty references, and zero duplicate article-summary pairs. Its length profile is very different from the bundled sample:

Statistic	XSum article tokens	XSum reference tokens	XSum reference compression
Count	500	500	500
Mean	401.7	21.1	9.3%
Median	306.0	21.0	6.8%
Standard deviation	304.2	5.3	8.7 pp
Minimum	19	3	0.8%
25th percentile	192.8	18.0	3.8%
75th percentile	517.3	24.0	11.1%
Maximum	1,743	48	89.5%

The XSum snapshot is therefore a harder stress test than the bundled sample: articles are roughly five times longer on average, while references stay almost the same length. This pushes the target compression from 29% down to 9%, so extractive three-sentence methods should be expected to lose precision unless the system adds length-aware sentence selection or an abstractive model.

4. Pipeline reflection

The end-to-end NLP workflow has six stages:

1. **Data validation** (`nlp_project.data`) — checks required columns, empty cells, summary-longer-than-article corruption; reports article/summary length statistics. Catches the most common dataset-replacement mistakes before training.
2. **Preprocessing** (`nlp_project.preprocess`) — HTML stripping, Unicode normalisation, boilerplate removal, regex sentence segmentation. Preserves casing and punctuation so downstream summarisers remain fluent.
3. **Summarisation** (`nlp_project.summarize`) — three interchangeable methods behind one function call. Abstractive BART is loaded lazily and skipped gracefully when the optional dependency is missing.

4. **Evaluation** (`nlp_project.evaluate`) — ROUGE-1/2/L/Lsum via the `rouge-score` library; per-row qualitative CSV; compression and timing metrics.
5. **Visualisation** (`nlp_project.visualize`) — ROUGE comparison, article and summary length histograms, compression-ratio bars, per-example ROUGE-1 distribution.
6. **Deployment** (`nlp_project.app`) — Streamlit demo with three tabs: paste-article, fetch-URL, and ROUGE evaluation refresh.

Two design choices proved decisive. **YAML-as-single-source-of-truth** keeps the CLI, tests, and the Streamlit app perfectly synchronised: one config edit changes the model, the metrics, and the deployed demo in lockstep. **Lazy abstractive loading** means CI runs in seconds on the cheap extractive baselines, while the same code path scales up to BART when a developer flips one flag.

5. Conclusion

We built a reproducible English news summarisation pipeline with three interchangeable summarisation methods (Lead-3, TextRank, BART), full ROUGE evaluation, and a Streamlit prototype that accepts pasted articles or public URLs. On the bundled sample, both extractive methods score around 0.36 ROUGE-1 with TextRank edging Lead-3 by ~1 point. With BART enabled and the full CNN/DailyMail dataset fetched, the same pipeline measures abstractive vs. extractive performance directly. Qualitative review identifies three failure modes that further work — chunked long-article inputs, length-aware decoding, and abstractive fine-tuning on XSum — would address.

The prototype demonstrates how NLP can support faster, more efficient news reading. It is functional, reproducible, and presentable, and it gives a foundation for future work on multi-document or aspect-based summarisation.

6. Team Contribution Statement

Member	Primary contributions
Nguyen Hoang Hieu (Ethan)	Dataset curation, web-scraping module (<code>nlp_project.scrapers</code>), CNN/DailyMail and XSum fetch scripts.
Thai Ba Hung	Text preprocessing (HTML cleaning, Unicode normalisation, sentence segmentation), exploratory length / vocabulary analysis.
Nguyen Quoc Dang	Summarisation methods (Lead-3, TextRank graph + PageRank, BART pipeline), method-selection logic and ablation runs.
Le Nguyen Gia Binh	ROUGE evaluation harness, all report figures, Streamlit demo, system integration, and final report preparation.

All four members co-authored this report, reviewed every merge into `main`, and rehearsed the presentation together.

7. Individual reflections (<= 250 words each)

Nguyen Hoang Hieu (Ethan): Data and scraping

My role focused on data collection, dataset organisation, and the scraping tools used by the demo. I helped prepare the bundled news sample, checked that each row followed the expected article-summary schema, and worked on `scripts/fetch_datasets.py` so the team could fetch CNN/DailyMail and XSum in a reproducible format. I also contributed to the web-scraping module that extracts article text from public URLs for the Streamlit app.

The biggest challenge was that real news pages contain a lot of non-article text, including navigation links, cookie messages, share prompts, and subscription banners. I learned that data collection is not just downloading text; it also requires provenance, schema consistency, and careful cleaning before modelling can be trusted. This project improved my ability to turn messy online text into a usable NLP dataset. In future work, I would add stronger source metadata, duplicate detection across outlets, and a larger cached benchmark set for more reliable evaluation.

Thai Ba Hung: Preprocessing and exploratory analysis

My role was preprocessing and exploratory analysis. I worked on HTML stripping, Unicode normalisation, boilerplate removal, and sentence segmentation. I also analysed article lengths, reference-summary lengths, compression ratios, and dataset balance so that the team could understand whether the sample data was suitable for summarisation experiments.

The main challenge was choosing cleaning rules that remove noise without damaging useful language. At first, it was tempting to treat summarisation like classification and aggressively lowercase or remove punctuation. I learned that this hurts readability and can weaken summarisation models because punctuation, casing, and sentence boundaries are important signals. I also learned how simple statistics can reveal important modelling problems, such as summaries being much shorter than generated extractive outputs. In future work, I would improve sentence splitting for edge cases, add better boilerplate detection for scraped pages, and include richer EDA such as vocabulary overlap and named-entity coverage between articles and summaries.

Nguyen Quoc Dang: Summarisation modelling

My role was summarisation modelling. I implemented and compared the Lead-3 baseline, TextRank extractive summariser, and the optional BART abstractive path. I helped design the method-selection logic so the project can run quickly offline with extractive methods but still support a stronger transformer model when the optional dependencies are installed.

The main challenge was that news summarisation baselines are surprisingly strong. Lead-3 often performs well because news articles usually place the most important information near the beginning. This made it harder for TextRank to show large improvements on the bundled sample. I learned that a good NLP model is not always the most complex one; it must match the dataset and task constraints. I also learned how to structure code so that expensive models are loaded lazily and do not break testing. In future work, I would experiment with length-aware TextRank, chunking for long articles, and BART decoding settings to improve precision on XSum-style short references.

Le Nguyen Gia Binh: Evaluation, visualisation, and demo

My role was evaluation, visualisation, system integration, and the Streamlit demo. I worked on the ROUGE evaluation harness, per-example qualitative review CSV, summary statistics, report figures, and the interactive interface where users can paste an article or fetch a URL and view a generated summary.

The biggest challenge was making the results interpretable rather than just producing metric files. Mean ROUGE scores are useful, but they hide per-example failures, length mismatch, and precision-recall trade-offs. I learned to connect quantitative evaluation with qualitative examples so the team could explain why a method succeeds or fails. I also learned that a small working demo makes the project much easier to communicate because users can test the NLP system directly. In future work, I would add confidence-style diagnostics, side-by-side method comparison in the app, clearer error messages for failed URL scraping, and automatic PDF/report generation with embedded figures.

8. References

1. See, A., Liu, P. J., & Manning, C. D. (2017). *Get To The Point: Summarization with Pointer-Generator Networks*. ACL.
2. Narayan, S., Cohen, S. B., & Lapata, M. (2018). *Don't Give Me the Details, Just the Summary! Topic-Aware Convolutional Neural Networks for Extreme Summarization*. EMNLP.
3. Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., & Zettlemoyer, L. (2020). *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*. ACL.
4. Lin, C.-Y. (2004). *ROUGE: A Package for Automatic Evaluation of Summaries*. Text Summarization Branches Out.
5. Mihalcea, R., & Tarau, P. (2004). *TextRank: Bringing Order into Texts*. EMNLP.
6. Hermann, K. M., Kočiský, T., Grefenstette, E., Espeholt, L., Kay, W., Suleyman, M., & Blunsom, P. (2015). *Teaching Machines to Read and Comprehend*. NeurIPS.

9. Appendix

A. Reproducing every number in this report

```
pip install -e ".[dev,app,data,transformer]"

# Bundled sample (no internet, no GPU required)
python -m nlp_project.cli run-all --config config/project_config.yaml

# Full CNN/DailyMail benchmark
python scripts/fetch_datasets.py --dataset cnn_dailymail --split test --max-rows 500
# (edit config/project_config.yaml: data.path -> data/cnn_dailymail.csv,
#  model.use_abstractive -> true)
python -m nlp_project.cli run-all --config config/project_config.yaml

# Interactive prototype
streamlit run src/nlp_project/app.py
```

The full pytest suite runs in ~4 seconds offline:

```
python -m pytest -q
```

B. Repository layout

config/	YAML config (single source of truth)
data/	bundled sample + dataset README
src/nlp_project/	
app.py	Streamlit demo
cli.py	command-line entry point
config.py	typed YAML loader
data.py	loading, validation, splits
evaluate.py	ROUGE evaluation
preprocess.py	HTML cleaning + sentence segmentation
proposal.py	proposal markdown + PDF
scraper.py	news URL fetcher / parser
summarize.py	Lead-3, TextRank, BART
visualize.py	every report figure
tests/	pytest suite (26 tests, offline)
scripts/	fetch_datasets.py, render_proposal_pdf.py
reports/	metrics, figures, final_report.md, presentation.md